

# Class 5: Advance Docker Concepts

## Session Overview

1. **Introduction to Docker Compose:**
  - Definition and use cases of Docker Compose.
  - Structure and components of a docker-compose.yml file.
  - Hands-on demo: Managing a multi-container application using Docker Compose.
2. **Networking in Docker:**
  - Explanation of Docker networking modes: Bridge, Host, and Overlay.
  - Configuring custom networks to enable container communication.
  - Practical examples: Networking containers for container-to-container communication.
3. **Storage in Docker:**
  - Overview of Docker storage concepts: Volumes, Bind mounts, tmpfs.
  - Best practices for managing persistent data in Docker containers.
  - Practical lab: Using Docker volumes for persistent data storage.
4. **Multi-Container Applications:**
  - How to deploy and manage applications consisting of multiple containers.
  - Practical example: Setting up and running a web server and database using Docker Compose.

### Prerequisites:

- Basic understanding of Docker and containerization.
- Knowledge of Docker commands for managing single containers.

## Introduction

## Docker Compose

**Docker Compose** is a tool that simplifies the process of defining and running multi-container Docker applications. With Docker Compose, you can define multiple services, networks, and volumes in a single configuration file (typically named docker-compose.yml) and manage them as a cohesive unit. This is particularly useful for applications that consist of multiple components, such as a web server, a database, and a caching system, which all need to run in separate containers but must communicate and work together.

Docker Compose allows developers to set up and manage complex environments with just a few commands, making it easier to develop, test, and deploy applications without needing to manually handle each container.

### Why Use Docker Compose?

In real-world applications, it's common to have multiple services running together. Managing each service individually with separate Docker commands can be cumbersome and prone to error. Docker Compose solves this by providing a declarative way to define and orchestrate multi-container applications. Key benefits of using Docker Compose include:

1. **Simplification of Multi-Container Setup:** Define multiple services, networks, and volumes in a single file (docker-compose.yml).
2. **Version Control:** The docker-compose.yml file can be version-controlled, providing consistency across environments (development, testing, production).
3. **Portability:** The setup defined in the Compose file is portable across systems. Running an entire multi-container application becomes as simple as using docker-compose up.
4. **Dependency Management:** Docker Compose can handle the order of container startup, ensuring that dependent services (e.g., a web app that depends on a database) start in the correct sequence.

### The Structure of a Docker Compose File (docker-compose.yml)

The docker-compose.yml file is the heart of Docker Compose. It's where you define the services, networks, and volumes required by your application. A typical Compose file includes the following sections:

1. **Version:** Specifies the version of Docker Compose being used.
2. **Services:** Defines the containers that need to be created. Each service corresponds to a container and includes information like the image to use, ports to expose, environment variables, and more.
3. **Networks:** Defines the custom networks that will be created for container communication. Containers in the same network can easily communicate with each other.
4. **Volumes:** Defines shared storage between containers or between a container and the host machine, ensuring data persistence.

A basic example of a docker-compose.yml file:

```
version: '3'
services:
  web:
    image: nginx
    ports:
      - "80:80"
    volumes:
      - ./web:/usr/share/nginx/html
    networks:
      - frontend
  db:
    image: postgres
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - db-data:/var/lib/postgresql/data
```

```
networks:
  - backend
volumes:
  db-data:
networks:
  frontend:
  backend:
```

In this example:

- Two services, web (a web server) and db (a PostgreSQL database), are defined.
- Volumes are used to persist the database data.
- Custom networks are set up for both frontend and backend communication.

### Key Commands in Docker Compose

Once your docker-compose.yml file is defined, Docker Compose provides several key commands to manage your multi-container application:

1. **docker-compose up:** This command starts all services defined in the docker-compose.yml file. If the images aren't already available, Docker Compose will pull them automatically.
2. **docker-compose down:** Stops and removes all running containers, networks, and volumes created by Docker Compose.
3. **docker-compose ps:** Lists the running services.
4. **docker-compose logs:** Shows logs from all running services.
5. **docker-compose stop:** Stops all running services without removing them.

### Practical Example of Using Docker Compose

Let's consider a scenario where you need to manage a simple web application consisting of a web server (Nginx) and a database (PostgreSQL). In the docker-compose.yml file:

```
version: '3'
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    volumes:
      - ./web:/usr/share/nginx/html
    networks:
      - app-network
  database:
    image: postgres
    environment:
      POSTGRES_USER: example
      POSTGRES_PASSWORD: example
    networks:
```

```
- app-network
networks:
  app-network:
```

To bring this application up, simply run:

```
docker-compose up
```

Docker Compose will automatically create the necessary containers, networks, and volumes.

#### Use Cases for Docker Compose

- **Microservices:** Managing multiple microservices, each in its own container, becomes easier with Docker Compose.
- **Development Environments:** Set up complex development environments with databases, message brokers, and caching systems quickly.
- **Testing and CI/CD Pipelines:** Use Docker Compose to run tests in isolated, reproducible environments.
- **Production Deployments:** Compose can be used for small-scale production deployments and can integrate with orchestration tools like Kubernetes for larger setups.

## Docker Networking

In Docker, networking plays a critical role in how containers communicate with each other and the outside world. Docker's networking features allow containers to be connected to each other, the host machine, or external networks seamlessly. Understanding Docker networking is essential for building and managing containerized applications, especially when deploying multi-container setups where different services (such as databases, web servers, or application layers) need to communicate. Docker provides several networking modes, each serving specific use cases. These modes include **Bridge**, **Host**, and **Overlay**, among others. Each networking mode has its own characteristics in terms of container-to-container communication, performance, and isolation.

#### Docker Networking Modes

1. **Bridge Network Mode (Default)**
  - **Bridge** is Docker's default networking mode, used when a container is created without specifying a custom network. In this mode, Docker creates a virtual network interface and connects all containers running on this network to an internal bridge (usually named `docker0`).
  - Containers within the same bridge network can communicate with each other using container names or IP addresses, but they are isolated from the host system and external networks.
  - **Use case:** Bridge mode is ideal for multi-container applications where internal services need to communicate without exposing them to the outside world.

Example:

```
docker run --name webapp --network bridge nginx
```

## 2. Host Network Mode

- In **Host** mode, Docker bypasses the network isolation between the container and the host. The container shares the host's network stack and can use the same IP address and port as the host. This results in improved performance since there is no additional layer for network translation.
- However, using the host's network stack can lead to conflicts, especially if multiple containers or the host are trying to bind to the same port.
- **Use case:** Host networking is useful when you need low-latency communication or when port conflicts are unlikely, such as in performance-critical applications.

Example:

```
docker run --name myapp --network host nginx
```

## 3. Overlay Network Mode

**Docker Swarm** is Docker's native orchestration tool for managing containerized applications across multiple nodes (servers). It allows you to cluster multiple Docker engines and manage them as a single system. Key features of Docker Swarm include service discovery, automatic load balancing, scaling, and rolling updates. It ensures high availability by redistributing tasks if a node fails.

### Key Concepts:

- **Swarm Mode:** Activates Docker Swarm functionality for clustering.
- **Nodes:** Machines in the swarm, categorized into manager and worker nodes.
- **Services:** Definitions of tasks (containers) running in the swarm.
- **Overlay Network:** Allows communication between services across nodes.
- **Load Balancing:** Automatically balances traffic across container replicas.

### Hands-on Steps:

1. **Initialize Swarm** on a manager node (`docker swarm init`).
2. **Add Worker Nodes** using a token generated from the manager (`docker swarm join`).
3. **Deploy Services** like `nginx` using `docker service create`.
4. **Scale Services** (`docker service scale`) to handle increased traffic.
5. **Perform Rolling Updates** by updating the service image (`docker service update`).

- **Overlay** networks are used in Docker's **swarm mode** for enabling multi-host networking. This mode allows containers running on different Docker hosts to

communicate securely as if they were on the same host. Overlay networks are useful for distributed applications deployed across multiple servers.

- It abstracts the complexity of cross-host communication and securely encrypts container traffic between different Docker hosts.
- **Use case:** Overlay networks are essential for running distributed applications, microservices, and clustered environments, especially in Docker Swarm or Kubernetes.

Example:

```
docker network create -d overlay my-overlay-network
```

#### 4. None Network Mode

- In **None** mode, Docker disables networking altogether for the container. This means the container is isolated from any network and can only interact with other containers or the host through IPC (Inter-Process Communication).
- **Use case:** None mode is typically used for security purposes or testing when you need to isolate a container completely from all network communication.

Example:

```
docker run --network none nginx
```

#### 5. Custom Networks

- Docker allows users to create custom networks using the `docker network create` command. Custom networks provide more control over how containers communicate with each other and can be created with specific drivers like **bridge** or **overlay**.
- When containers are connected to the same custom network, they can communicate by name rather than IP address, which is especially useful when using Docker Compose to manage multi-container applications.

Example of creating a custom bridge network:

```
docker network create my-custom-network
docker run --name app1 --network my-custom-network nginx
docker run --name app2 --network my-custom-network redis
```

### Configuring Custom Networks for Docker Containers

When configuring custom networks, you can define specific network settings such as IP address ranges, subnet masks, and gateways. This flexibility allows you to create isolated or interconnected environments for your containers, depending on your application needs.

**Example: Creating a Custom Bridge Network**

```
docker network create --driver bridge \
  --subnet=192.168.1.0/24 \
```

```
--gateway=192.168.1.1 my-bridge-network
```

In this example, a custom bridge network named my-bridge-network is created with a specified subnet and gateway. Containers connected to this network will have IP addresses in the 192.168.1.0/24 range.

#### Example: Connecting Containers to a Custom Network

```
docker run --name app1 --network my-bridge-network nginx
docker run --name app2 --network my-bridge-network redis
```

By connecting the containers to the same network, they can communicate with each other using their container names:

```
ping app1
```

#### Container-to-Container Communication

When containers are on the same custom network, they can communicate with each other using their service or container names as hostnames. For example, a web server container can communicate with a database container using its name instead of relying on hardcoded IP addresses. This dynamic hostname resolution allows for more flexible and scalable applications. In Docker Compose, specifying services in the same network makes them discoverable by each other. For instance, in a multi-container setup using Docker Compose:

```
version: '3'
services:
  web:
    image: nginx
    networks:
      - app-network
  database:
    image: postgres
    networks:
      - app-network
networks:
  app-network:
```

Both the web and database services are in the app-network, allowing them to communicate seamlessly.

### Practical Example of Networking in Docker

To illustrate the power of Docker networking, let's set up a simple multi-container application using a web server (Nginx) and a Redis cache server. We'll place both containers in the same network so they can communicate with each other.

```
# Create a custom network
docker network create app-network

# Run an Nginx container in the custom network
docker run --name webapp --network app-network -d nginx

# Run a Redis container in the same custom network
docker run --name cache --network app-network -d redis

# Now, webapp can communicate with cache using the Redis service name
```

This setup enables the two containers to communicate through the app-network using their service names (webapp and cache).

## Docker Storage

Storage in Docker is essential for managing data within containers. Unlike traditional virtual machines, Docker containers are designed to be ephemeral, meaning they can be stopped and removed at any time. This ephemerality means that any data stored within a container's filesystem is lost when the container is removed. To handle persistent data, Docker provides several mechanisms, including **Volumes**, **Bind Mounts**, and **tmpfs** mounts. Each type of storage has specific use cases and characteristics, making it important to choose the right one based on the needs of your application.

### Docker Storage Types

#### 1. **Volumes**

- **Volumes** are the preferred way to persist data in Docker. They are managed by Docker and stored outside of the container's filesystem. Volumes are stored in a part of the host filesystem which is managed by Docker (`/var/lib/docker/volumes/` on Linux). They are designed to be shared and reused among containers and are suitable for use cases requiring persistent or shared storage.
- **Features:**
  - **Isolation:** Volumes are isolated from the container filesystem.
  - **Data Persistence:** Data in volumes persists even when the container is removed.



- **Backup and Restore:** Volumes can be easily backed up and restored.
- **Sharing:** Volumes can be shared between multiple containers.
- **Use Case:** Storing database files, application logs, and configuration files.

#### Creating and Using Volumes:

```
# Create a volume
docker volume create my_volume

# Run a container with a volume
docker run -d --name my_container -v my_volume:/data nginx
```

### 2. Bind Mounts

- **Bind Mounts** allow you to mount a directory from the host filesystem into a container. This means that changes made to the data in the host directory are reflected in the container and vice versa. Bind mounts are useful when you need to work with files on the host system or when you need to share files between the host and the container.
- **Features:**
  - **Direct Access:** Provides direct access to host files and directories.
  - **Data Persistence:** Changes made in the mounted directory are persistent and visible to the host.
  - **Flexibility:** Useful for development when you need to edit files on the host and see changes in real time.
- **Use Case:** Development environments, configuration files, or when integrating with host file systems.

#### Creating and Using Bind Mounts:

```
# Run a container with a bind mount
docker run -d --name my_container -v /host/path:/container/path nginx
```

### 3. tmpfs Mounts

- **tmpfs** mounts provide temporary storage that is stored in the host system's memory rather than on disk. This type of storage is useful for applications that require fast access to temporary data or for data that should not persist beyond the lifetime of the container.
- **Features:**
  - **In-Memory Storage:** Data is stored in RAM and lost when the container stops or restarts.
  - **Speed:** Provides faster read/write operations compared to disk-based storage.
  - **Ephemeral:** Suitable for data that doesn't need to be persisted across container restarts.
- **Use Case:** Temporary caching, session storage, or sensitive information that shouldn't be written to disk.

#### Creating and Using tmpfs Mounts:

```
# Run a container with a tmpfs mount
docker run -d --name my_container --tmpfs /tmp:rw,size=100m nginx
```

### Best Practices for Managing Docker Data

1. **Use Volumes for Persistent Data:** Whenever you need to ensure data persists beyond the life of a container or needs to be shared among multiple containers, use Docker volumes.
2. **Use Bind Mounts for Development:** Bind mounts are excellent for development environments where you need live updates to code or configuration files without rebuilding containers.
3. **Leverage tmpfs for Performance:** For data that is temporary and needs fast access, use tmpfs mounts. This is particularly useful for caching or ephemeral storage needs.
4. **Backup and Restore Volumes:** Regularly backup important volumes to prevent data loss. Docker volumes can be backed up by copying the volume directory from the host filesystem or by using Docker's built-in tools.
5. **Clean Up Unused Volumes:** Docker volumes can accumulate over time, especially if containers are frequently created and removed. Use `docker volume prune` to clean up unused volumes and free up space.
6. **Consider Data Security:** For sensitive data, ensure that volumes and mounts are properly secured and that only authorized containers and users have access.

### Practical Example of Docker Storage

To demonstrate Docker storage, let's set up a simple example using a volume to persist data for a MySQL database.

1. **Create a Volume for MySQL Data:**

```
docker volume create mysql_data
```

2. **Run a MySQL Container Using the Volume:**

```
docker run -d --name mysql -e MYSQL_ROOT_PASSWORD=rootpassword -v
mysql_data:/var/lib/mysql mysql:latest
```

Here, the `mysql_data` volume is mounted to `/var/lib/mysql` inside the container, ensuring that database files are persisted even if the container is removed or recreated.

3. **Inspect the Volume:**

```
docker volume inspect mysql_data
```

This command provides details about the volume, including its mount point on the host system.

# Multi-Container Applications in Docker

In modern application development, many applications consist of multiple components or services that interact with each other. For example, a typical web application might have separate components for the frontend, backend, and database. Docker makes it easy to deploy and manage multi-container applications by using Docker Compose, which allows you to define and run multi-container Docker applications with a single configuration file.

## Why Multi-Container Applications?

Multi-container applications are crucial for several reasons:

- **Separation of Concerns:** Different parts of the application can be developed, updated, and scaled independently.
- **Modularity:** Services can be updated or replaced individually without affecting the entire application.
- **Scalability:** Individual components can be scaled horizontally according to their load.
- **Resilience:** If one component fails, it does not necessarily bring down the entire application.

## Using Docker Compose

Docker Compose is a tool that simplifies the management of multi-container applications. It uses a YAML file (docker-compose.yml) to define the services, networks, and volumes that are part of the application. This file allows you to configure and start multiple Docker containers with a single command.

## Basic Structure of a docker-compose.yml File:

```
version: '3'
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    networks:
      - my-network

  database:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
    volumes:
      - db-data:/var/lib/mysql
    networks:
      - my-network
```

```
networks:
  my-network:
    driver: bridge

volumes:
  db-data:
```

### Key Sections in the docker-compose.yml File:

1. **services:** Defines the different services (containers) that make up the application. Each service is given a name (e.g., web, database) and configuration details such as the image to use, ports, environment variables, and volumes.
2. **networks:** Specifies the networks to which services should be connected. Docker Compose allows you to create and manage custom networks.
3. **volumes:** Defines named volumes that can be shared among services. Named volumes persist data even if the containers are stopped or removed.

### Practical Example: Web Server and Database

Let's set up a simple multi-container application with a web server (Nginx) and a MySQL database.

1. **Create a docker-compose.yml File:**

Create a file named docker-compose.yml in your project directory with the following content:

```
version: '3'
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    networks:
      - my-network

  database:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
    volumes:
      - db-data:/var/lib/mysql
    networks:
      - my-network

networks:
  my-network:
    driver: bridge

volumes:
```

```
db-data:
```

## 2. Start the Multi-Container Application:

Run the following command in the directory where your docker-compose.yml file is located:

```
docker-compose up -d
```

The -d flag runs the containers in detached mode.

## 3. Verify the Application:

- **Check Running Containers:**

```
docker-compose ps
```

- **Access the Web Server:**

Open your browser and navigate to <http://localhost:8080>. You should see the default Nginx welcome page.

- **Check the Database:**

You can use a MySQL client to connect to the database. The database is running on the default port 3306, and you can access it using the credentials specified in the docker-compose.yml file.

## 4. Stop the Application:

To stop and remove the containers, networks, and volumes defined in the docker-compose.yml file, run:

```
docker-compose down
```

## Benefits of Multi-Container Applications with Docker Compose

- **Simplified Configuration:** Docker Compose allows you to define all services, networks, and volumes in a single file.
- **Easier Management:** Start, stop, and manage multiple containers with simple commands.
- **Consistent Environment:** Ensure that the application runs in a consistent environment, irrespective of where it is deployed.
- **Scalability:** Easily scale services up or down by adjusting the docker-compose.yml file and using the appropriate commands.

# Hands-On Lab

## 1. Create Node.js Application:

- Create a new directory for the project:

```
mkdir my-node-app
```

```
cd my-node-app
```

- Initialize a new Node.js project:

```
npm init -y
```

- Install Express and Mongoose:

```
npm install express mongoose
```

- Create a server.js file:

```
const express = require('express');
const mongoose = require('mongoose');

const app = express();
const port = 3000;

mongoose.connect('mongodb://mongo:27017/mydatabase', { useUnifiedTopology: true, useNewUrlParser: true });

app.get('/', (req, res) => {
  res.send('Hello from Node.js and Docker!');
});

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}/`);
});
```

## 2. Create Dockerfile for Node.js Application:

- Create a Dockerfile in the my-node-app directory:

```
# Use the official Node.js image
FROM node:14

# Set the working directory
WORKDIR /usr/src/app
```

```
# Copy package.json and package-lock.json
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the rest of the application code
COPY . .

# Expose the port
EXPOSE 3000

# Start the application
CMD ["node", "server.js"]
```

3. **Create Docker Compose File:**
  - In the same directory, create a docker-compose.yml file:

```
version: '3'
services:
  web:
    build: .
    ports:
      - "3000:3000"
    networks:
      - my-network
    depends_on:
      - mongo
  mongo:
    image: mongo
    volumes:
      - mongo-data:/data/db
    networks:
      - my-network
networks:
  my-network:
```

```
driver: bridge
```

```
volumes:  
  mongo-data:
```

4. **Build and Start the Application:**

- Run the following command to build and start the containers:

```
docker-compose up --build
```

5. **Verify Deployment:**

- Open a web browser and navigate to <http://localhost:3000>. You should see “Hello from Node.js and Docker!”.

6. **Stop and Remove Containers:**

- Stop and remove the containers with:

```
docker-compose down
```

### 3. Docker Networking

**Objective:**

- Configure and test Docker networking between containers.

**Steps:**

1. **Inspect Custom Network:**

- After running `docker-compose up`, inspect the custom network:

```
docker network inspect my-network
```

2. **Test Container Communication:**

- Enter the web container:

```
docker exec -it <web_container_id> /bin/sh
```

- Test connectivity to the MongoDB container:

```
apk add --no-cache curl  
curl mongo:27017
```



3. **Verify Network Configuration:**

- Ensure that containers are connected to the my-network network.

#### 4. Docker Storage

**Objective:**

- Manage Docker storage using volumes and bind mounts.

**Steps:**

1. **Use Named Volumes:**

- The docker-compose.yml file already defines a named volume mongo-data. Verify its existence:

```
docker volume ls
```

2. **Bind Mounts:**

- Modify the docker-compose.yml file to include a bind mount for the Node.js application logs:

```
services:
  web:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - ./logs:/usr/src/app/logs
    networks:
      - my-network
    depends_on:
      - mongo
```

3. **Apply Changes:**

- Restart the application to apply the changes:

```
docker-compose up -d
```

4. **Verify Bind Mount:**

- Check the logs directory in your project for any log files created by the Node.js application.

## 5. Multi-Container Application (30 minutes)

### Objective:

- Deploy and manage a multi-container application with a Node.js frontend and a MongoDB backend.

### Steps:

1. **Expand Docker Compose File:**
  - Add additional services if needed, such as a Redis cache, and ensure all services are correctly linked in the docker-compose.yml file.
2. **Deploy the Expanded Application:**
  - Run:

```
docker-compose up -d
```

3. **Test Interaction Between Containers:**
  - Test the interactions between your Node.js application and MongoDB. For example, you could add CRUD operations to your Node.js application that interact with the MongoDB database.
4. **Cleanup:**
  - Stop and remove all containers:

```
docker-compose down
```